

Compressing Icelandic name declension patterns into a 3.27 kB trie

August 2, 2025

Displaying personal names in Icelandic user interfaces is surprisingly hard. This is because of *declension* — a language feature where the forms of nouns change to communicate a syntactic function.

In Icelandic, personal names have four forms, one for each of the [grammatical cases of Icelandic nouns](#). Take the name “*Guðmundur*”:

Grammatical case	Form
Nominative	Guðmundur
Accusative	Guðmund
Dative	Guðmundi
Genitive	Guðmundar

When including a name in a sentence, the sentence’s structure determines the grammatical case, and correspondingly, a certain form of the name should be used. Using the wrong form results in a “broken” feel that native speakers associate with non-native speakers not yet fluent in the language.

The problem is that Icelandic personal names are always stored in the **nominative** case. If you’ve loaded a user from a database, their name will be in the nominative

example, the accusative form of the name.

As a developer, you can work around that by rewriting the sentence to use the nominative case, which can be very awkward, or by using a pronoun (e.g. *they*). Both are unsatisfactory.

A few years ago, I built a JavaScript library to solve this issue. It applies any of the four grammatical cases to an Icelandic name, provided in the nominative case:

```
applyCase("Guðmundur", "accusative")  
//⇒ "Guðmund"
```



When building this library, I did not code *any* declension rules by hand. Instead, the rules of Icelandic name declension are derived from public Icelandic data for personal names and their forms. The rules are encoded in a trie-like data structure that uses clever compression techniques to get the library’s bundle size under 4.5 kB gzipped. This lets the library be included in web apps without increasing bundle size significantly.

The rest of the post will walk through this problem in detail, and go over the compression techniques I used to get the trie to such a small size.

Data for Icelandic name declension

Iceland has a publicly run institution, *Árnastofnun*, that manages the **Database of Icelandic Morphology** (DIM). The database was created, amongst other reasons, to support Icelandic language technology.

DIM publishes various **datasets**, but we’ll use **Kristín’s Format** (the K-format), downloadable as a CSV. Here’s what the K-format data entries for “Guðmundur” look like:

```

Guðmundur;355264;kk;ism;1;;;K;Guðmund;þFET;1;;;
Guðmundur;355264;kk;ism;1;;;K;Guðmundi;þGFET;1;;;
Guðmundur;355264;kk;ism;1;;;K;Guðmundar;EFET;1;;;
^^^^^^^^^^          ^^^^^^^^^^  ^^^^
Name                  Form         Case

```

From this, we can see that the name “Guðmundur” in the accusative (þFET) case is “Guðmund”, and so on.

From the K-format data, we can construct an array for each name containing its form for each grammatical case:

```

[
  "Guðmundur", // Nominative
  "Guðmund",   // Accusative
  "Guðmundi",  // Dative
  "Guðmundar", // Genitive
]

```



However, the K-format has data for most words in the Icelandic language, not just personal names. With over **7 million** entries, this data set is huge. We’ll need some way to whittle the list down.

Luckily for us, Iceland has the [Personal Names Register](#). It lists all Icelandic personal names approved — and rejected — by the [Personal Names Committee](#) (yes, that exists).

We can use the set of approved Icelandic names to filter the K-format data. Of the roughly 4,500 approved Icelandic names, the K-format has declension data for over 3,600. With that, we have declension data for more than 80% of Icelandic names:

```

const NAME_FORMS = [
  [

```



```
    "Aðalberg",  
    "Aðalbergs"  
  ],  
  [  
    "Agnes",  
    "Agnesi",  
    "Agnesi",  
    "Agnesar"  
  ],  
  // ...and over 3,600 more  
]
```

Naive implementation

With the declension data in place, let's get to writing our library. The library will export a single `applyCase` function that takes a name in the nominative case and the grammatical case that the name should be returned in:

```
function applyCase(name: string, grammaticalCase: Case) {  
  // ...  
}
```

```
applyCase("Guðmundur", "accusative")  
//⇒ "Guðmund"
```

The naive implementation would be to find the forms of the name and the index of the form to return:

```
const CASES = ["nominative", "accusative", "dative", "genitive"];  
  
function applyCase(name: string, grammaticalCase: Case) {  
  const nameForms = NAME_FORMS.find(forms ⇒ forms[0] === name);  
  const caseIndex = CASES.indexOf(grammaticalCase);
```

and, with those in hand, return the form at `caseIndex` if `nameForms` was found for the input `name`, otherwise returning `name` as a fallback:

```
function applyCase(name: string, grammaticalCase: Case) {  
  const nameForms = NAME_FORMS.find(forms => forms[0] === name);  
  const caseIndex = CASES.indexOf(grammaticalCase);  
  return nameForms?.[caseIndex] || name;  
}
```



This “works” but has two main issues, the first of which is bundle size. The `NAME_FORMS` list is about 30 kB gzipped, which I think is a tad much to add to a web app’s bundle size.

The second issue is that this naive implementation only works for names in the `NAME_FORMS` list. As mentioned earlier, there are around 800 approved Icelandic names that are not covered by the DIM data.

Let’s see how we can solve both of those.

Encoding the forms compactly

We’re currently storing the four forms of each name in full. We can remove a lot of redundancy by finding the **longest common prefix** of the name and the suffixes of each form.

Consider the forms of “Guðmundur”:

Guðmundur
Guðmund
Guðmundi



The longest common prefix is “Guðmund”, and the suffixes are as follows:

```
Guðmund ur
Guðmund
Guðmund i
Guðmund ar
^^^^^^ ^^
Prefix  Suffix
```



We can store the suffixes compactly in a string like so:

```
suffixes.join(",")
```



Which for Guðmundur, gives us:

```
"ur,,i,ar"
```



Since `applyCase` receives the nominative case of the name as input, we can derive the prefix from the length of the nominative suffix’s length.

```
function getPrefix(nameNominative, suffixLength) {
  return nameNominative.slice(0, -suffixLength);
}
```



```
const suffixes = "ur,,i,ar";
const nominativeSuffix = suffixes.split(",")[0];
```

We'll call this method of encoding the suffixes of each form in a string the “suffix encoding”, or just “encoding”, from here on.

A feature of the suffix encoding is that the encoding is not tied to any specific name (“Guðmund” appears nowhere). Instead, the suffix encoding describes a *pattern* of declension, which we'll use to our advantage later.

Retrieving the suffixes by name

When we were storing the raw forms in an array, it was very easy to find the forms of any given name:

```
NAME_FORMS.find(forms => forms[0] === name)
```



But the suffix encoding doesn't encode the name itself, so we need a way to retrieve the encoding. The simplest method would be a plain hash map:

```
const nameToFormsEncoding = {  
  Guðmundur: "ur,,i,ar",  
  // ...3,600 more lines  
};
```



Putting bundle size concerns aside, a hash map doesn't solve the problem of names not in the list of approved Icelandic names being excluded.

Here, one helpful fact about Icelandic declension is that names with similar suffixes *tend* to follow the same pattern of declension. These names ending in “ur” all have

Ástvaldur
Bárður
Freymundur
Ingimundur
Sigurður
Þórður



There are, in fact, 88 approved Icelandic names with this exact pattern of declension, and they all end with “*dur*”, “*tur*” or “*ður*”.

The naive approach, then, would be to implement a `getSuffixEncoding` function that captures these patterns:

```
function getSuffixEncoding(name) {  
  if (/((d|ð|t)ur$/.test(name)) {  
    return "ur,,i,ar";  
  }  
  // ...  
}
```



But that quickly breaks down. There are other names ending with “*ður*” or “*dur*” that follow a different pattern of declension:

- “*Aðalráður*” and “*Arnmóður*” have a suffix encoding of `"ur,,i,s"`
- “*Baldur*” has a suffix encoding of `"ur,ur,ri,urs"`
- “*Hlöður*” and “*Lýður*” both have a suffix encoding of `"ur,,,s"`

In fact, take a look at this [gist](#) showing every approved Icelandic personal name grouped by their suffix encoding (there are 124 unique encodings). You’ll immediately find patterns, but if you take a closer look you’ll find numerous

exceptions in code would be a tedious and brittle affair.

Instead of trying to code up the rules manually, we can use a data structure that lends itself perfectly to this problem.

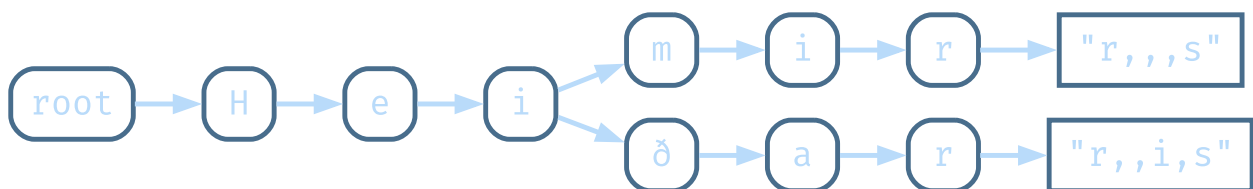
Tries

The **trie** data structure, also known as a prefix tree, is a tree data structure that maps string keys to values. In tries, each character in the key becomes a node in the tree that points to the next possible characters.

Take, for example, the name “*Heimir*”, which has a suffix encoding of “*r,,s*”. If we create an empty trie and insert “*Heimir*” and “*r,,s*” as a key-value pair into it, we get:



Let’s now insert “*Heiðar*” into the trie, which has a suffix encoding of “*r,,i,s*”. The names share the first three characters, so they share the first three nodes in the trie:



However, we actually want to insert the keys *backwards* into the trie. That is because, like I mentioned earlier, names with similar endings (suffixes) tend to have similar suffix encodings. Inserting keys backwards results in the values for all names sharing a certain suffix being grouped within that suffix’s subtree.

and their encodings.

Ylfur ur,i,i,ar

Knútur ur,,i,s

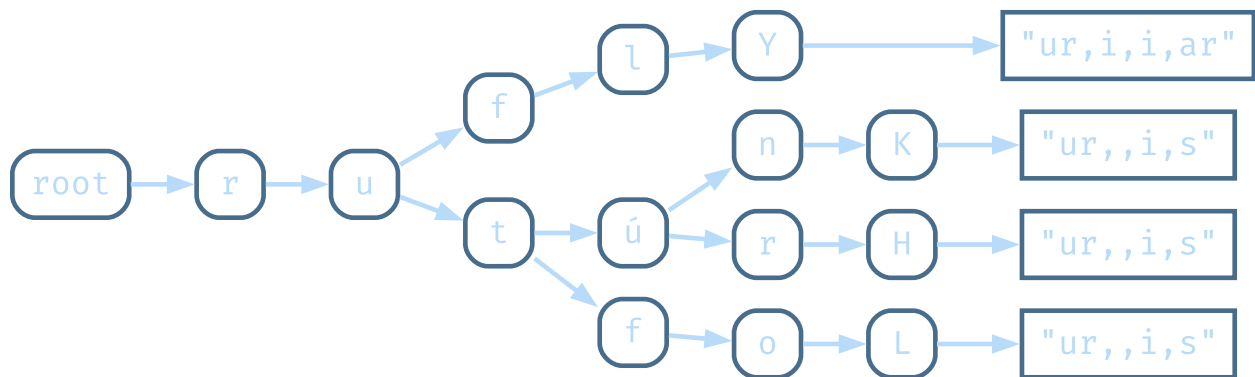
Hrútur ur,,i,s

Loftur ur,,i,s

Name Suffix encoding



Inserting them *backwards* into a new trie gives us the following:



Once we start inserting the names backwards, every node in the trie corresponds to a specific suffix match:

- The $\boxed{r} \rightarrow \boxed{u}$ subtree corresponds to the “ur” suffix.
- The $\boxed{r} \rightarrow \boxed{u} \rightarrow \boxed{t}$ subtree corresponds to the “tur” suffix.

Additionally:

- The $\boxed{r} \rightarrow \boxed{u}$ subtree contains the values for all names ending in “ur”.
- The $\boxed{r} \rightarrow \boxed{u} \rightarrow \boxed{t}$ subtree contains the values for all names ending in “tur”.

help us find patterns in data to value mappings. We can then apply those patterns to not-before-seen names.

Before we get to that, let's quickly cover trie lookups.

Trie lookups

Let's implement a `trieLookup` function that takes the trie's `root` node and a `key` (name) to find a value for:

```
interface TrieNode {
  children?: { [key: string]: TrieNode };
  value?: string;
}

function trieLookup(root: TrieNode, key: string) {
  // ...
}
```



For each character in the `key`, we traverse to the `child` node for that character, stopping if no such node exists. After that, we return the value of the resulting node, if present:

```
function trieLookup(root: TrieNode, key: string) {
  let node: TrieNode | undefined = root;
  for (const char of reverse(key)) {
    node = node.children?.[char];
    if (!node) {
      break;
    }
  }
  return node?.value;
}
```



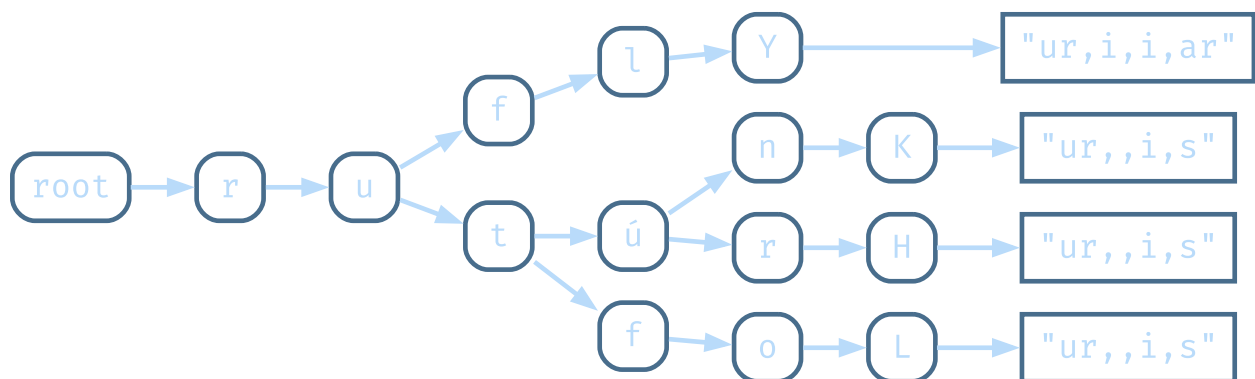
Looking up a name that we insert into the trie returns its suffix encoding, as expected:

```
trieLookup(root, "Loftur")  
//⇒ "ur,,i,s"
```

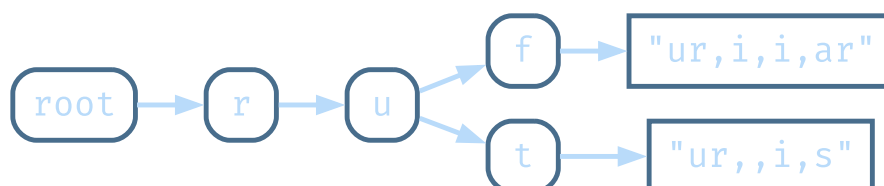


Compressing the trie

In our trie from earlier, every leaf in the `r` ▶ `u` ▶ `t` subtree has the same value of `"ur,,i,s"`:



When every leaf in a subtree has a common value, we can *compress* the subtree. We do that by setting the value of the subtree's root to the value of its leaves, and then deleting every child of the root.



The trie from above, compressed.

this operation.

```
function compress(node: TrieNode): string | null {  
    // ...  
}
```



The `compress` function should return `null` and do nothing if `node`'s children do not share a single common value. If they *do* share a common value, it should delete all of its children and assign their common value to itself.

The first step is to collect the values of `node`'s children by invoking `compress` recursively (using a **depth-first** traversal):

```
const values = Object.values(node.children).map(compress);
```



If there is not a single shared value, we return `null`:

```
if (new Set(values).size === 1 || values[0] = null) {  
    return null;  
}
```



Otherwise, we assign the value to `node`, remove the children, and return the value.

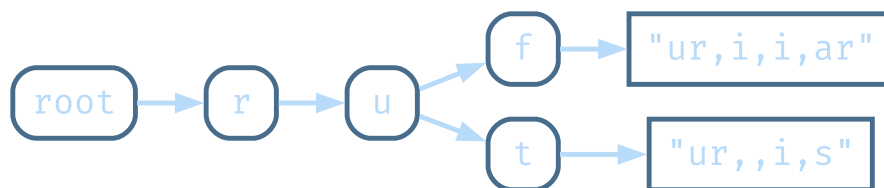
```
node.value = values[0];  
node.children = {};  
return node.value;
```





```
function compress(node: TrieNode) {  
  const values = Object.values(node.children).map(compress);  
  values.push(node.value);  
  
  if (new Set(values).size === 1 || values[0] = null) {  
    return null;  
  }  
  
  node.value = values[0];  
  node.children = {};  
  return node.value;  
}  
compress(root);
```

Let's take a second look at the compressed trie:



After compression, it communicates the following information:

- All names ending in “*fur*” resolve to a value of `ur,i,i,ar`
- All names ending in “*tur*” resolve to a value of `ur,,i,s`

When we originally inserted “*Ylfur*” into the trie, the associated value was stored under `r` → `u` → `f` → `l` → `Y`, but after compressing the trie, only the `r` → `u` → `f` part of that path remains.

This means that our `trieLookup` function from earlier will return `null` for “*Ylfur*”:

```
let node: TrieNode | undefined = root;
for (const char of reverse(key)) {
  node = node.children?.[char];
```

'node' will be null for 'f→l'

```
  if (!node) {
    break;
  }
}
return node?.value;
}
```

```
trieLookup(root, "Ylfur")
//⇒ null
```

We can fix that by returning the value of the last node we encountered:

```
function trieLookup(root: TrieNode, key: string) {
  let node = root;
  for (const char of reverse(key)) {
    const next = node.children?.[char];
    if (!next) {
      break;
    }
    node = next;
  }
  return node.value;
}
```

```
trieLookup(root, "Ylfur")
//⇒ "ur,i,i,ar"
```

We only override `node` if there is a `next` node.

Now, looking up the original four input names returns the values for those names:

```
trieLookup(trie, "Knútur") //⇒ "ur,,i,s"  
trieLookup(trie, "Hrútur") //⇒ "ur,,i,s"  
trieLookup(trie, "Loftur") //⇒ "ur,,i,s"
```

However, we also get values for lookup keys not in the original input data:

```
trieLookup(trie, "Bjartur")  
//⇒ "ur,,i,s"
```



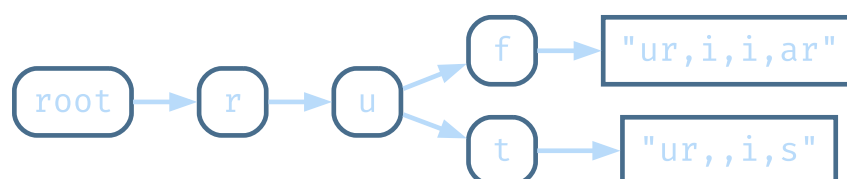
This was not the case prior to compressing the trie — only the original input keys returned a value in the original trie.

Lookups in the compressed trie return

- "ur,i,i,ar" for all lookup keys matching *fur , and
- "ur,,i,s" for all lookup keys matching *tur .

The compressed trie has, in some sense, “learned” the suffix patterns of the input data, and returns values based on that.

Names in the input data ending in *tur always resolved to the same value so the (r) ➔ (u) ➔ (t) subtree was compressed — same with *fur . However, there were multiple values for names ending in *ur so the tree diverges after (r) ➔ (u):



^tʊɹ ɪɪɔɪ ʌtʊɹ .

“*Sakur*” is one such key. When invoking `trieLookup` the last hit `node` is the `u` node. Since `u` has no value, `null` is returned:

```
trieLookup(trie, "Sakur")  
//⇒ null
```



If every key in the trie’s input data ending in `*ur` were to resolve to the same value, then “*Sakur*” should resolve to that value. However, not every key ending in `*ur` resolves to the same value — keys ending in `*tur` resolve to one value and keys ending in `*fur` to another.

For a key matching `*ur` but not `*(t|f)ur`, we *could* just pick one of the branches. However, at most one of the branches resolves to the correct value (and in many cases, none of the branches do). The natural conclusion, then, is to *not* return a value.

The compressed trie acts as a sort of suffix-to-value pattern matcher. If a certain suffix in the input data always maps to a certain value, the compressed trie always returns that value for keys matching the suffix. But for “ambiguous” suffix matches, no value is returned.

Since Icelandic names with similar suffixes *tend* to have the same pattern of declension, the theory is that the compressed trie should be able to predict the correct pattern of declension for not-before-seen names. Let’s see how well that theory holds.

Compressing 3,600 names

Inserting those names and their suffix encodings into a new trie gives us a trie with 10,284 nodes, 3,638 of which are leaves. Compressing the trie by merging subtrees with common values reduces the total number of nodes to 1,588. Of those, 1,261 are leaves and 327 are not.

	Uncompressed	Compressed	Compressed (%)
Total nodes	10,284	1,588	15.4%
Non-leaf nodes	6,646	327	4.9%
Leaf nodes	3,638	1,261	34.6%

Compressing the trie resulted in 6,319 non-leaf nodes being removed, which is **over 95%**.

The removal of non-leaf nodes means shorter paths from the root to the leaves of the trie. Here’s a chart showing the traversal depth of lookups for the keys in the input data for the compressed and uncompressed tries:

Lookup depth correspond to the length of the suffix match needed for a value to be returned. For the majority of names in the original input data, that length is three or

Testing the trie on not-before-seen names

In testing how well the compressed trie predicts the declension patterns of not-before-seen names, the 800 approved Icelandic names that we don't have declension data for serve as good test cases.

I wrote a function to pick 100 of those names at random and (manually) categorized the declension pattern returned when looking those names up in the trie:

Result	Count
Perfect (declension applied)	62
Perfect (no declension applied)	12
Should have applied declension	23
Wrong, should not be declined	2
Wrong declension	1

This gives us a rough indication that, for not-before-seen Icelandic names, the compressed trie gives us correct results 74% of the time and wrong results 26% of the time.

The *"Should have applied declension"* case, which constitutes 23% of results, results in `applyCase` not applying declension to the name and returning it as-is. That result is wrong, but I consider it a lesser kind of wrong.

Still, these are just 100 random names. Some names are far more common than others. It'd be more interesting to see how well the compressed trie performs for the most common names.

Luckily for us, [Statistics Iceland](#) publishes data on how many individuals have **specific names**. Using that data, I created the chart below. It shows the number of people holding each name in the approved list of names as a first name. The 3,600

declension data are colored red.

☐ Logarithmic

Note: Since relatively few names dominate this list, I made the chart logarithmic by default. You can use the toggle in the upper-right corner to make it linear.

363,314 people hold a name from the approved list of Icelandic names as a first name. Of those, 5,833 have names that don't have declension data available.

As we can see from the chart, the commonality of names is far from evenly distributed. In fact, the top 100 names without declension data are held by 4,990 people. Those 4,990 people constitute 86% of the 5,833 people that hold one of the 800 names without declension data available.

I went ahead and categorized the declension results for those 100 names, multiplying the result by the number of people holding the name:

Result	Number of people
--------	------------------

Perfect (no declension applied)	440
Should have applied declension	915
Wrong, should not be declined	101
Wrong declension	45
Total	4,990

1,061 wrong results gives us an error rate of 21%. If we extrapolate that 21% error rate across the 5,833 people holding names without declension data available, we get 1,240 wrong results. Dividing 1,240 wrong results by the 363,314 people holding names in the approved list of Icelandic names gives us an error rate of 0.34%.

If we do the same math with only the names that were *incorrectly* declined, we get an error rate of 0.046%.

Regularity and comprehensiveness

The compressed trie captures the rules of Icelandic name declension to an impressive degree. I attribute this to the *regularity* and *comprehensiveness* of the data on Icelandic name declension, where

- *regularity* is the degree to which similar key suffixes map to the same values, and
- *comprehensiveness* is how well the input data captures rules *and* exceptions to them.

Regularity

If the input data were *irregular* — meaning that there's no significant relationship between suffixes and associated values — the values of leaves in subtrees would frequently differ. That would prevent subtree compression, resulting in a not-very-compressed trie that is similar, if not identical, to the original trie. The less a trie is compressed, the longer the suffix match needs to be for a value to be returned.

more frequently compressed, leading to shorter suffix matches being required for values to be returned.

Comprehensiveness

Subtrees are only ever incorrectly compressed if the original trie lacks a counterexample to the regularity that led to compression. If a counterexample had been present, it would have prevented compression and created an exception to the rule.

If we pick, say, 450 Icelandic names at random, we will capture many of the rules of Icelandic name declension, and some counterexamples to them. Still, 450 names are only about 10% of approved Icelandic names, so we can expect loads of declension rules *not* to be covered by that sample.

But with over 3,600 samples, as in our case, we have over 80% coverage. With data that comprehensive, the compressed trie captures the rules — and exceptions to those rules — to an impressive degree.

Bundle size

I've mentioned bundle time a few times — let's finally measure it!

I measured the size of storing the declension data for the 3,600 names that we have declension data for in the following ways:

- List (the `NAME_FORMS` list from before)
- Trie (uncompressed)
- Trie (compressed)

Here are the results:

Trie (uncompressed)
14.47 kB gzipped (66.68 kB minified)

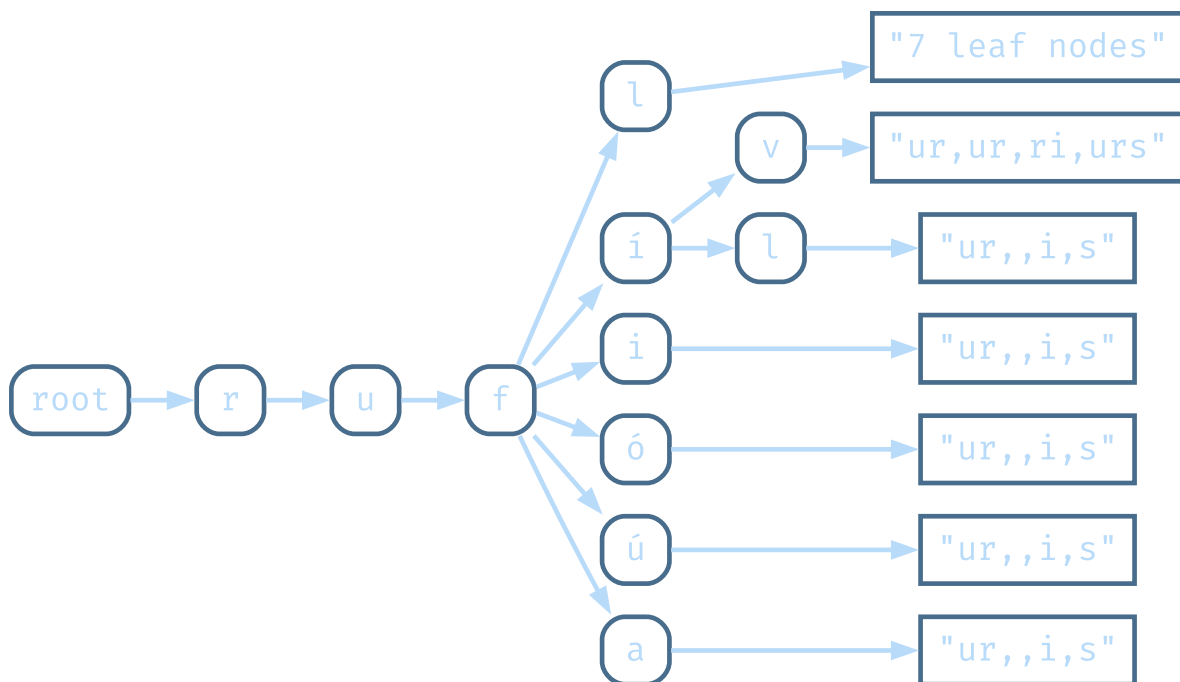
Trie (compressed)
4.01 kB gzipped (14.41 kB minified)

Note: The trie is serialized to a compact string representation to make its size smaller (see [serializer](#) and [deserializer](#)). For comparison, the compressed trie represented as JSON is 4.75 kB.

4.01 kB is very compact, but we can take the compression one step further.

Merging sibling leaves with common suffixes

Take a look at the `r` `u` `f` subtree from the compressed trie — it represents names matching `*fur`:



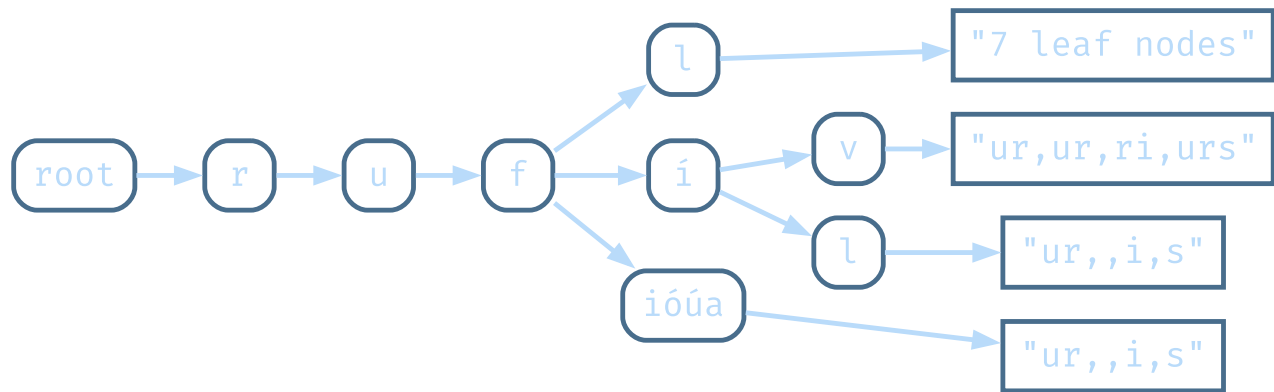
Note: I've hidden the full `*lfur` subtree to simplify this view.

The `í`, `ó`, `ú`, `a` sibling leaves following `r` `u` `f` all resolve to the same value of `ur,,i,s`. However, the `l` and `i` subtrees have

being compressed.

What we can do here is merge sibling leaves with common values. That results in the

(i), (ó), (ú), (a) leaves being merged into a single (ióúa) leaf node:



Let's implement a `mergeLeavesWithCommonValues` function that performs this compression.

```
function mergeLeavesWithCommonValues(node: TrieNode) {
  // ...
}
```



Firstly, if the `node` has no children, we can immediately return, otherwise performing the operation recursively on the children:

```
if (!node.children) {
  return;
}

for (const child of Object.values(node.children)) {
  mergeLeavesWithCommonValues(child)
}
```



1. The child is a leaf node with a `value` .
2. The child is a non-leaf node.

We want to merge leaf nodes with the same value, so we'll group the keys of leaf nodes by their value:

```
const keysByValue: Record<string, string> = {};
```



However, we want to leave non-leaf nodes alone, so we'll define a new `newChildren` object to place them into as we encounter them:

```
const newChildren: Record<string, TrieNode> = {};
```



With those defined, we'll iterate through the children, transferring non-leaf nodes immediately and grouping leaf keys by values:

```
for (const [key, child] of Object.entries(node.children)) {  
  const isLeaf = !!child.value;  
  if (isLeaf) {  
    keysByValue[child.value] ??= [];  
    keysByValue[child.value].push(key)  
  } else {  
    newChildren[key] = child;  
  }  
}
```



When looking at this, one could be concerned that a `child` might contain both a value *and* children. In our Icelandic names trie, however, there is no overlap because each name in the input data starts with an uppercase character.

line 30.

```
for (const [value, keys] of Object.entries(keysByValue)) {  
  newChildren[keys.join("")] = { value };  
}  
node.children = newChildren;
```



This concludes the implementation. The full implementation is a bit long, so I won't show it in full here — you can view it in this [gist on GitHub](#).

We need to consider merged keys in our `trieLookup` function. To do that, we'll update the `trieLookup` function to use a new `findChild` function instead of `node.children?.[char]` when finding the next node.

```
function trieLookup(root: TrieNode, key: string) {  
  let node = root;  
  for (const char of reverse(key)) {  
    const next = findChild(node, char);  
    if (!next) {  
      break;  
    }  
    node = next;  
  }  
  return node.value;  
}
```



Implementing `findChild` is relatively simple: we iterate through the children, returning the current child if its key contains the lookup character:

```
function findChild(node: TrieNode, char: string) {  
  const children = node.children || {};
```



```
    if (key.includes(char)) {  
        return child  
    }  
}  
}
```

It's worth mentioning that, unlike merging subtrees with common values, merging sibling leaves has no functional effect on the trie. This layer of compression is purely to make the trie's footprint smaller.

Trie after merging sibling leaves

Here is the node count table from before with a new column that shows the results for the trie that has also had its sibling leaves merged:

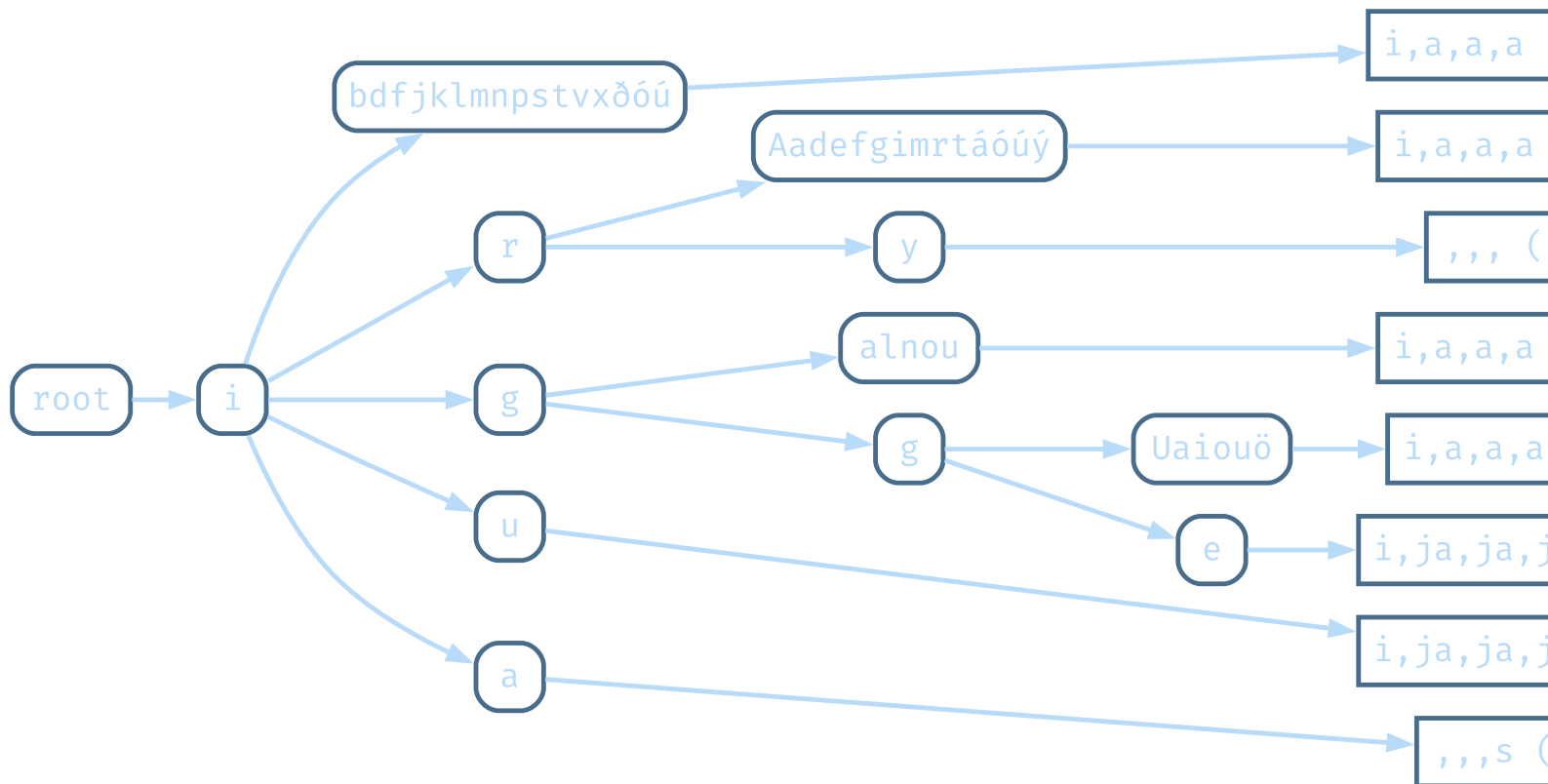
	Uncompressed	Only subtrees merged	Subtrees and sibling leaves merged
Total nodes	10,284	1,588	972
Non-leaf nodes	6,646	327	327
Leaf nodes	3,638	1,261	645

Merging sibling leaf nodes with common values almost cuts the number of leaf nodes in half! Since we're only touching the leaf nodes, the number of non-leaf nodes stays the same. Lookup depth is also not affected.

One interesting statistic is how many names in the original input data each leaf node now represents. Here are the top 50 leaf nodes by the number of names they represent:

that indicates that Icelandic names ending in *i* exhibit a high degree of regularity in their pattern of declension.

Let's take a closer look at the **i** subtree. Next to each value node, I've added the number of names that the leaf node represents in parentheses.



The **i** subtree is built from 223 names starting with “i”. Only four of those names don’t follow the declension pattern of “i,a,a,a”. That’s a really high degree of regularity!

Those four names serve as important counterexamples to the general rule that names ending in “i” have a suffix encoding of “i,a,a,a”. Without them, the **i** subtree would have been compressed to a single value node.

Final bundle size

the trie.

List

30.17 kB gzipped (152.48 kB minified)

Trie (uncompressed)

14.47 kB gzipped (66.68 kB minified)

Trie (subtrees merged)

4.01 kB gzipped (14.41 kB minified)

Trie (subtrees and leaves merged)

3.27 kB gzipped (9.3 kB minified)

It saves us 0.74 kB. That's a small number in absolute terms, but hey, it's an 18% improvement!

The beygla library

I use the compressed trie in a declension library for Icelandic names called **beygla**. The library is 4.46 kB gzipped, 3.27 kB of which is the serialized trie. As described, it exports an `applyCase` function that is used to apply grammatical cases to Icelandic names.

The beygla library is used, for example, by the Icelandic judicial system to **decline the names of defendants** in indictments.

The library includes a `"beygla/addresses"` module (see motivating issue). It uses the exact same approach, with that module's trie being built from data on Icelandic addresses.

Trading bundle size for 100% correctness

The indictment example I linked above uses the **strict version** of beygla:

The `"beygla/strict"` module only applies cases to names in the approved list of Icelandic names. I added it after [this issue](#) was raised:

*"We are using beygla in a project within the public sector. Our users care **a lot** about using grammatically correct Icelandic."*

When first developing beygla, I cared *a lot* about the bundle size being as small as possible so that Icelandic web apps could use the library without being concerned about JavaScript bloat. I found the compressed trie really powerful in that it both made the library *tiny* while also applying declension to not-before-seen names with few errors. There's certainly a cool factor to it.

But still, beygla does occasionally produce a wrong result, which is *not* an appropriate trade-off in contexts such as generating indictments.

`"beygla/strict"` is about 15 kB gzipped (10 kB more than the default beygla module), which, honestly, is not that large of a bundle size increase.

Because of that, if I were developing the library again today, I probably would have made `"beygla/strict"` the default. For apps willing to trade 100% correctness for bundle size, they could opt for the less-but-mostly-correct 5 kB variant. Perhaps I'll publish a new major version of beygla with that change soon.

Note: The `beygla/strict` module encodes the list of approved Icelandic names in *another* trie using a compact string serialization. The [implementing PR](#) describes how that trie is serialized, so I won't cover it here.

Final words

Building beygla was a super fun problem to solve. When I first started the project, I didn't expect to be able to get the bundle size so low. The compressed trie ended up being really effective for encoding Icelandic declension patterns.

There are many languages with declension as a language feature (such as Slavic and Balkan languages), so there is an opportunity to apply the ideas explored in this post to those languages. Native speakers of said languages are well suited to explore that.

I'd like to thank [Eiríkur Fannar Torfason](#) and [Vilhjálmur Thorsteinsson](#) for reading and providing feedback on draft versions of this post. Vilhjálmur actually identified an optimization opportunity in `beygla` that reduced the size of the trie from 3.43 kB to 3.27 kB ([see PR](#)).

Thanks for reading, I hope this was interesting.

— Alex Harri

Mailing list

To be notified of new posts, subscribe to my mailing list.

Alex Harri

© 2026 Alex Harri Jónsson

Links

-  [GitHub](#)
-  [LinkedIn](#)
-  [RSS](#)

Pages

- [Home](#)
- [Blog](#)
- [About](#)
- [Snippets](#)

The monospace font on this website is [MonoLisa](#), courtesy of [FaceType](#).